Data Models

Multi-Model Capstone

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

Building On: Topics 1–10

- **Topics 2–5:** Relational model — FK constraints, normalisation, SQL joins, ACID transactions
- **Topic 6:** Document model — aggregate reads, structural variation, snapshot discipline
- **Topic 7:** XML/XSD — schema-validated exchange at organisational boundaries
- **Topics 8–9:** RDF — URI-based identity, cross-source integration, SPARQL
- **Topic 10:** Property graph — bounded traversal, edge properties, cycle detection

Each model works inside its own territory. What happens when a real system needs **all five**?

The Super-App on One Database

A startup builds the entire ride-hailing super-app on a single relational database. At launch, it works — ride bookings, payments, driver settlements all run under ACID.

```
Workload                  │                Relational (MySQL)                  │
                          │                                                    │
Ride booking ──────────►  │  √ ACID transactions, FK constraints  (cheap path) │
+ payments                │                                                    │
                          │                                                    │
Personalisation ──────►   │  X 7-table JOIN, EAV, 400 ms          (expensive)  │
                          │                                                    │
Regulatory ────────────►  │  X No XSD validation, 38 invoices      (expensive) │
reporting                 │    rejected                                        │
                          │                                                    │
Partner ───────────────►  │  X Ad-hoc ETL, breaks on format        (expensive) │
integration               │    changes                                         │
                          │                                                    │
Fraud monitoring ──►      │  X Recursive CTE, 47 seconds,          (expensive) │
                          │    string-hack cycle prevention                    │
```

Four workloads on the expensive path. One model doing five jobs.

Four Expensive Paths: The Symptoms

Workload	Symptom	Topic Reference
Personalisation	7-table JOIN, EAV tables, 400 ms profile load	Topic 6 Failure 1
Regulatory reporting	38 invoices rejected — element ordering violates `xs:sequence`	Topic 7 Failure
Partner integration	Ad-hoc ETL breaks on every format change	Topic 8 Failure 1
Fraud monitoring	Recursive CTE, 47 seconds, string-hack cycle prevention	Topic 10 Failure 1

Each failure traces to the same root cause: the workload's dominant access pattern does not match the model's strengths. The relational model is not wrong — it is wrong **for that specific workload**.

Failure 2: Right Models, No Boundaries

A more experienced team uses relational for transactions, MongoDB for profiles, Neo4j for fraud detection. Right models, right workloads.

But they design each store **in isolation**.

```
Relational          MongoDB          Neo4j

 Rides              Profiles         Fraud
 Payments                            Graph
 Settle-
 ments

   ⚠️ no arrows       ⚠️ no arrows      ⚠️
```

After 6 months: **12% of driver earnings summaries** in MongoDB diverge from relational settlement records. **1,200 graph edges** point to non-existent accounts. Two weeks debugging, one-off reconciliation script.

The Pattern Across Both Failures

Failure	What went wrong	What was missing
Single-model trap	Four workloads forced onto one model's expensive path	Workload decomposition + model assignment
Right models, no boundaries	Data silently diverges across three isolated stores	Boundary design — triggers, consistency, failure modes

Predict: The startup had five workloads on one database. The experienced team used three models with no boundaries. What **five capabilities** must a multi-model design provide to prevent both failures?

Five Requirements

#	Requirement	Traces to
1	**Decompose** into workloads — access pattern, consistency, data shape	Failure 1: collapsing hides the selection decision
2	**Assign** a model per workload — two-part justification (cheap + substitute cost)	Failure 1: wrong model's expensive path forced
3	**Identify boundaries** — trigger, source of truth, transformation, consistency, failure mode	Failure 2: undesigned boundaries caused drift
4	**State the trade-off** at each boundary — what is gained, what is lost	Failure 2: polyglot cost must be acknowledged
5	**Produce an architecture diagram** — stores, workloads, data-flow arrows, consistency annotations	Both: the architecture must be visible

> Therefore, the engineer must **decompose by workload, assign by model fitness, design the boundary crossings, and state the trade-offs honestly** — not discover after deployment that five services cannot share one database and three stores cannot share data without design.

Workload: Not a Service, Not a Feature

> A **workload** is a characterised access pattern over a data store, defined by four properties: shape, dominant operations, consistency requirement, and growth pattern.

Property	Definition	Example
Shape	Structure of data as accessed	Normalised rows; nested document; ordered tree
Dominant ops	Top 2–3 reads, top 1–2 writes	"Load full profile on app open"
Consistency	What must be true after each operation	ACID isolation; eventual convergence
Growth	What grows and at what rate	Rows; nesting depth; edge count

A workload is not a service. Ride booking and payment processing both require ACID on normalised rows — **same workload**, different services.

Model–Workload Fitness

> **Model–workload fitness** is the degree to which a model's native operations match a workload's dominant operations. High fitness = cheap path. Low fitness = workarounds.

Every model has a **cheap path** (what it was designed to make fast) and an **expensive path** (what requires workarounds).

The assignment question is not "which model is best?" but "**which model's cheap path aligns with this workload's dominant operations?**"

> **Predict:** The ride-booking workload's dominant operations are "create ride + assign driver + record fare" (atomic write) and "check promo usage limit" (constraint check). Which model's cheap path aligns? What would the document model sacrifice?

Fitness Answer: The Relational Cheap Path

The **relational model**. ACID atomicity, FK constraints, CHECK predicates — all on the cheap path.

Cost of substituting the document model:

- Cross-document atomicity becomes application-managed (not engine-enforced)
- Promo usage limit — no CHECK constraint; the app must verify
- Referential integrity — no FK safety net; orphaned references possible

Cost: correctness risk on every write that the relational engine prevents for free.

Data-Flow Boundary

A **data-flow boundary** is a point where data produced in one model's store is consumed by another. Each boundary is defined by five properties.

Property	Definition	Example
Trigger	Event that initiates the flow	Ride completion; monthly batch
Source of truth	Which store owns the authoritative version	Relational owns ride records
Transformation	What changes between source and target	Row → document subdocument
Consistency contract	Guarantee at the boundary	Synchronous; eventual; batch
Failure mode	What happens when target is unavailable	Retry queue; idempotent replay

The boundary is where the complexity of multi-model architecture **concentrates**.

One Entity, Five Shapes: Driver Ahmad Ibrahim

The same driver appears in five stores with five identifiers:

Model	Ahmad's identifier	Format
Relational	`driver_id = 5582`	Auto-increment integer
Document	`userId: "driver-5582"`	Application-assigned string
Property graph	`id: 'DRV-5582'`	Canonical string
RDF	`ex:driver_drv5582`	URI
XML/XSD	`SG-S1234567A`	National registry ID

Five stores. Five identifiers. One person. Every boundary must resolve the translation.

Dataset Scale Across Five Models

Model	Core entities	Source
Relational	2 drivers, 3 passengers, 2 active rides, 2 payments	Topic 2–5
Document	7 user profiles (3 passengers, 2 drivers, 2 merchants)	Topic 6
XML/XSD	1 sample UBL e-invoice per driver per settlement period	Topic 7
RDF	Driver URIs, zone URIs, transit-authority licence triples	Topic 8–9
Property graph	60 accounts, 150 transfers, 2 embedded fraud rings	Topic 10

Small enough to trace by hand. Large enough that each model's data has realistic shape.

Ahmad: Relational (Transactional Core)

Normalised tables, ACID guarantees, FK constraints (Topics 2–5):

Drivers

drv_id	name
5582	Ahmad Ibrahim
5583	Priya Nair

Rides

ride_id	drv_id	fare
2001	5582	68.50
2002	5583	18.50

Payments

payment_id	ride_id	amount
3001	2001	68.50
3002	2002	18.50

DriverSettlements

drv_id	balance	last_settled
5582	0.00	2024-03-14

The source of truth for money. Every ride references a valid driver; every payment references a valid ride.

Ahmad: Document (Personalisation)

A single nested document in MongoDB — one `findOne` returns everything (Topic 6):

```
{
  "userId": "driver-5582",
  "displayName": "Ahmad Ibrahim",
  "vehicle": { "make": "Toyota", "model": "Prius", "year": 2022 },
  "license": { "licenseNumber": "S1234567A", "expiryDate": "2026-08-15" },
  "preferences": { "maxDistance": 25, "preferredZones": ["central", "east"] },
  "earnings": { "currentWeek": 1240.50,
    "incentiveProgress": {"ridesCompleted": 42, "target": 50, "bonus": 80.00}
  },
  "subscriptionSnapshot": {
    "commissionRateAtSignUp": 0.20,
    "signUpDate": "2024-01-15"
  }
}
```

Notice the identifier shift: `driver_id = 5582` became `userId: "driver-5582"`. And `commissionRateAtSignUp` (0.20) does not change when the plan's current rate changes — that is **snapshot discipline**.

Ahmad: Property Graph and RDF

Property graph (fraud monitoring, Topic 10): Ahmad is an `:Account` node. Only transfers above SGD 1,000 create edges:

```
(PAX-1001: Sarah Tan) ──[SGD 68.50 ride_payment 2024-03-15]──▶ (DRV-5582: Ahmad Ibrahim)
```

RDF (partner integration, Topic 8–9): Ahmad is a URI node linked to external identifiers:

```
ex:driver_drv5582  ex:transitLicence  "SG-S1234567A" ;
                   ex:operatesIn      ex:zone_central , ex:zone_east .
```

Ahmad: XML/XSD (Regulatory)

XML/XSD (regulatory reporting, Topic 7): Ahmad's settlements generate a Peppol e-invoice at month-end:

```
<cac:AccountingSupplierParty>
  <cbc:EndpointID schemeID="SG:NRIC">SG-S1234567A</cbc:EndpointID>
</cac:AccountingSupplierParty>
```

Identifier: national registry ID `SG-S1234567A` — the only **externally assigned** ID. The relational system knows `driver_id = 5582`; the XML gateway knows `SG-S1234567A`. The boundary must translate.

The Integration Tax

Same person, five identifiers. Every boundary must map between them.

Boundary	Mapping	Fragility
Relational → Document	`5582` → `"driver-5582"`	Naming convention; breaks if integer changes
Relational → Property graph	`5582` → `'DRV-5582'`	Pipeline prefix; orphaned if ID renumbered
Relational → RDF	`5582` → `ex:driver_drv5582`	URI embeds integer; link breaks on change
Relational → XML	`5582` → lookup `licence_no` → `SG-S1234567A`	**Insulated** — external ID, independent

Predict: The relational team migrates `driver_id` from auto-increment integer to UUID. Which stores break, and which are insulated?

Integration Tax: Answer

Three of four boundaries break.

- **Document:** Every profile referencing `driver-5582` points to a non-existent relational ID
- **Property graph:** Nodes with `id: 'DRV-5582'` are orphaned — new edges use UUID-based IDs, old traversals hit dead ends
- **RDF:** URIs embedding `drv5582` lose their `ex:relationalId` link

XML boundary is unaffected — it uses `SG-S1234567A`, the transit authority's ID, independent of the internal scheme.

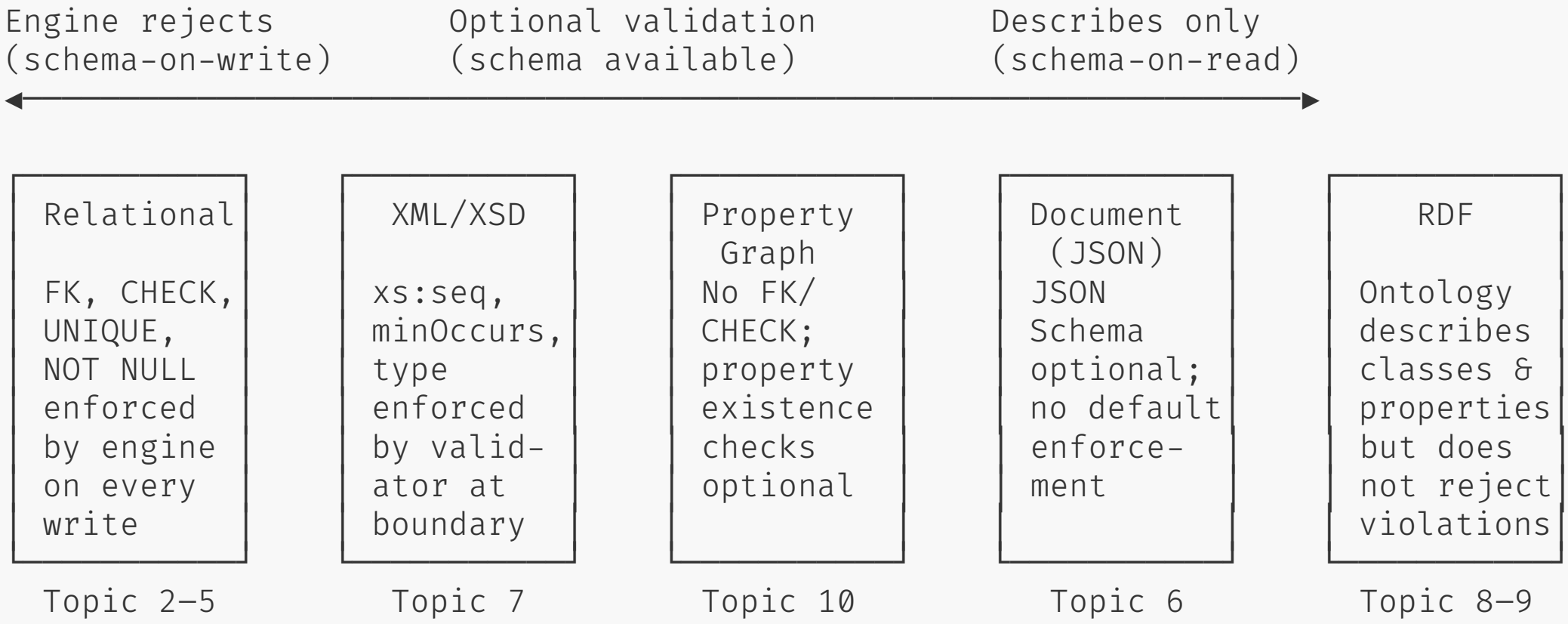
Mitigation: Design a canonical identifier independent of relational storage format. Or accept that ID migrations in a multi-model system require coordinated updates across all stores.

Five Models: The Capstone Comparison

Model	Native strength	Native weakness	Ch
Relational	ACID, FK/CHECK/UNIQUE, normalised storage, ad-hoc joins	Structural variation (EAV), aggregate reads (multi-JOIN), bounded traversal (recursive CTE)	2–5
Document	Aggregate reads, structural variation, single-doc atomicity	Cross-doc joins (`$lookup`), multi-doc txns (optional), referential integrity (app-managed)	6
XML/XSD	Schema-validated exchange at org boundary, ordered tree, contract enforcement	Storage efficiency, query flexibility, over-engineered for internal use	7
RDF	Cross-source integration, open-world tolerance, URI identity, SPARQL federation	Transactions, performance, schema enforcement (describes only), edge properties (reification)	8–9
Property graph	Bounded traversal, edge properties, cycle detection, path queries	ACID multi-entity txns, constraint enforcement, ad-hoc aggregation	10

Every entry references a specific mechanism from a prior chapter — not opinions, but demonstrated capabilities.

The Enforcement Spectrum: All Five Models



The spectrum is not a ranking. The enforcement position must match the workload's consistency requirement.

Cheap Path vs Expensive Path: Concrete Operations

Operation	Cheap Model	How (Cheap)	Expensive Model	How (Expensive)
Atomic multi-step write	Relational	`BEGIN; 4 statements; COMMIT`	Document	4 collections × separate writes; multi-doc txn optional
Load variable profile	Document	`findOne` — 1 read, complete JSON	Relational	4-table JOIN + re-nest to JSON
Validate against external schema	XML/XSD	`xmllint --schema` — validator checks all	Document	No element ordering in JSON; portal rejects non-XML
Merge independent org datasets	RDF	URI alignment, automatic merge	Relational	Ad-hoc ETL, hardcoded ID mappings, breaks on changes
3-hop filtered traversal	Property graph	Cypher `MATCH` — 6 lines	Relational	Recursive CTE — ~20 lines, string-hack cycles

Predict: Pick any row and trace what happens if you use the **expensive** model for that operation in production. What is the specific cost — latency, developer hours, correctness risk?

Aligning Models to Workloads

Every expensive-path row traces to the same cost pattern: latency (400 ms profile load, 47-second traversal), correctness risk (no FK safety net, no schema validation), or developer hours (20-line recursive CTE vs 6-line Cypher). The fix is not to optimise the workaround — it is to **move the workload to the model whose cheap path matches**.

Compare with Slide 3 — the same five workloads, now each on its **cheap path**:

Workload	Model	Path	
Ride booking + payments	Relational — ACID, FK, CHECK	(Topic 2–5)	√ cheap
Personalisation	Document — aggregate reads, no JOINs	(Topic 6)	√ cheap
Regulatory reporting	XML/XSD — schema-on-write at org boundary	(Topic 7)	√ cheap
Partner integration	RDF — URI identity, SPARQL federation	(Topic 8–9)	√ cheap
Fraud monitoring	Property graph — bounded traversal, cycles	(Topic 10)	√ cheap

Five workloads, five models — each on its cheap path. The cost of this improvement: the **boundaries** between models.

Boundary Consistency: The Ride-Completion Event

A single ride completes. Four stores need data. Which crossings should be ACID, which eventual?

Boundary	Trigger	Consistency	Acceptable Delay	Failure Mode
Relational → Document	COMMIT event	Eventual	~2 seconds	Retry queue; ride committed regardless
Relational → Property Graph	Payment > SGD 50	Eventual	Minutes	Retry with idempotent replay
Relational → XML/XSD	Month-end batch	Batch	~1 month	Local validation before submission
Relational → RDF	Nightly ETL	Batch	~1 day	Partial ingestion; retry next cycle

Each boundary's consistency contract matches the workload's **actual tolerance** — not an aspirational ideal. This is the operational cost of polyglot persistence.

When Fewer Models Is the Right Answer

Not every system needs five data models.

- No regulatory reporting mandate? → **Remove XML/XSD** — correct design, not a gap
- No cross-organisational integration? → **Remove RDF** — complexity without benefit
- Profile workload small enough? → **Relational with denormalisation** may suffice

The question is not "which five models should I use?" but "**which workloads does this system actually have, and which resist the model I'm already using?**"

> **Predict:** A startup builds a food-delivery app with ride booking, payments, and user profiles — but no regulatory reporting and no partner integrations. How many data models? Which would you exclude, and why?

Fewer Models: Answer

Three models — possibly two.

- **Relational:** Transactional core (ride booking, payments) — ACID, FK, CHECK
- **Document:** Personalisation (profiles) — aggregate reads, structural variation
- **No XML/XSD:** No external schema mandate — no workload to justify the boundary
- **No RDF:** No cross-source integration need — no workload for URI alignment
- **No property graph:** No bounded-traversal workload — no fraud monitoring requirement

If the profile workload is small, denormalised relational queries may serve it. Two models, not three. Every model excluded **reduces boundary complexity**.

Tracing the Ride-Completion Event: Step 1

The event: Sarah Tan (passenger-1001) completes ride R-2001 with Ahmad Ibrahim (driver-5582). Fare: SGD 1,250. Promo RIDE50 applied (3 of 5 max).

Step 1 — The relational transaction (Topics 2–5):

Pre-transaction state: `PromoUsage(promo_id=1, code='RIDE50', times_used=3, max_uses=5)` | `DriverSettlements(drv_id=5582, balance=0.00)`

```
BEGIN;
INSERT INTO Rides (ride_id, passenger_id, driver_id, fare, ride_date, status)
  VALUES (2001, 1001, 5582, 68.50, '2024-03-15', 'completed');
INSERT INTO Payments (payment_id, ride_id, amount, method, payment_date)
  VALUES (3001, 2001, 68.50, 'card', '2024-03-15');
UPDATE PromoUsage SET times_used = times_used + 1
  WHERE promo_id = 1 AND times_used < max_uses;  -- 3 → 4, still < 5
UPDATE DriverSettlements SET balance = balance + (68.50 * 0.80)
  WHERE driver_id = 5582;  -- 0.00 → 54.80
COMMIT;
```

Four tables, one transaction, three constraint types. The relational cheap path.

Ride-Completion: Step 2 (Relational → Document)

Sarah's pre-ride MongoDB profile:

```
{
  "userId": "passenger-1001",
  "name": "Sarah Tan",
  "plan": "premium-corporate",
  "rewardPoints": 2840,
  "recentRides": [
    { "rideId": "R-1998", "fare": 45.00, "date": "2024-03-10", "driver": "Li Wei" },
    { "rideId": "R-1999", "fare": 78.50, "date": "2024-03-12", "driver": "Priya
Nair" }
  ]
}
```

Reward rate: 1 point per SGD 10 spent, rounded down.

> **Predict:** What should Sarah's profile look like after ride R-2001 (fare SGD 68.50)? Which fields change? What value should `rewardPoints` have?

Step 2: Profile Update

After COMMIT, an async event triggers a MongoDB update to Sarah's profile:

```
db.users.updateOne(
  { userId: "passenger-1001" },
  { $push: { recentRides: {
      rideId: "R-2001",
      fare: 68.50,
      date: "2024-03-15",
      driver: "Ahmad Ibrahim"    // snapshot — not a live reference
    }},
    $inc: { rewardPoints: 6 }  // 2840 → 2846
  }
)
```

The `driver: "Ahmad Ibrahim"` is a **snapshot** — Topic 6's snapshot discipline. If Ahmad later changes his display name, Sarah's historical ride record still reads "Ahmad Ibrahim." Consistency: **eventual** (~2 seconds). If MongoDB is unavailable, the event is queued — the ride is not rolled back.

Steps 3–5: Graph, XML, RDF

Step 3 — Fraud-detection edge (payment > SGD 50):

```
MERGE (from:Account {id: 'PAX-1001'})
  ON CREATE SET from.name = 'Sarah Tan', from.type = 'Personal'
MERGE (to:Account {id: 'DRV-5582'})
  ON CREATE SET to.name = 'Ahmad Ibrahim', to.type = 'Personal'
CREATE (from)-[:TRANSFERRED_TO {
  amount: 68.50, date: date('2024-03-15'), transfer_type: 'ride_payment'
}]→(to)
```

Step 4 — E-invoice (monthly batch):

```
<Invoice>
  <InvoiceNumber>INV-2024-03-DRV5582</InvoiceNumber>
  <Customer>
    <Name>Ahmad Ibrahim</Name>
    <CompanyID>SG-S1234567A</CompanyID>  <!—— national registry ID ——>
  </Customer>
  <InvoiceLines>
    <InvoiceLine>
      <ItemName>Driver Settlement — March 2024</ItemName>
      <Quantity>1</Quantity>
      <LineAmount currencyID="SGD">258.50</LineAmount>
    </InvoiceLine>
  </InvoiceLines>
  <MonetaryTotal>
    <PayableAmount currencyID="SGD">258.50</PayableAmount>
  </MonetaryTotal>
  <PaymentMeans><PaymentMethodCode>30</PaymentMethodCode></PaymentMeans>
</Invoice>
```

Element ordering satisfies `xs:sequence`; `PaymentMethodCode 30` satisfies enumeration. Identifier: `driver_id 5582` → lookup → `SG-S1234567A`. SGD 258.50 is Ahmad's accumulated March 2024 settlements (all rides at 80% commission) — the batch aggregates across the month, not just ride R-2001.

Step 5 — Partner graph (nightly ETL):

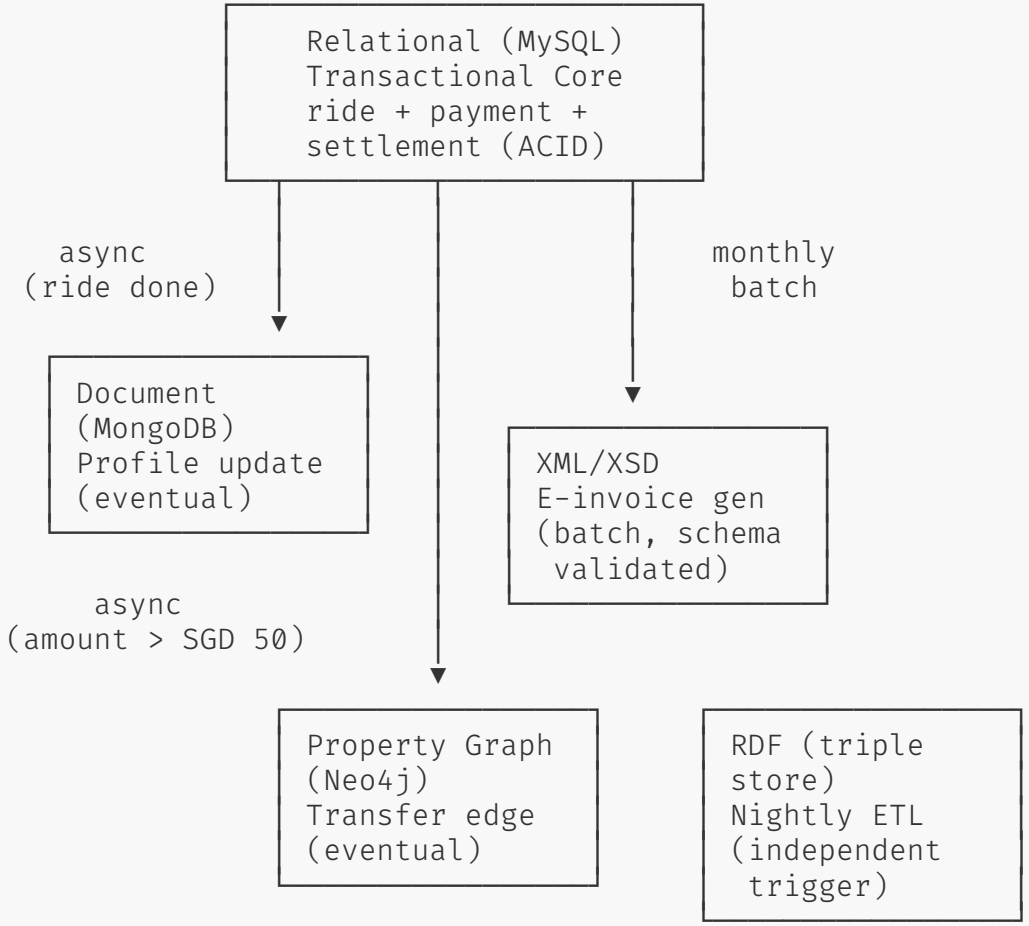
```
ex:driver_drv5582  ex:internalId      "DRV-5582" ;
                   ex:relationalId    5582 ;
                   ex:operatesIn      ex:zone_central , ex:zone_east .

ta:licence_SG_S1234567A  ta:expiryDate  "2026-08-15"^^xsd:date ;
                         ta:vehicleClass "Standard" .

ex:driver_drv5582  ex:transitLicence  "SG-S1234567A" .
```

Independent trigger — not ride-completion driven. `driver_id 5582` → URI `ex:driver_drv5582`. Nightly staleness: a driver added today appears in the partner graph tomorrow.

Complete Data Trace: One Event, Five Stores



One business event, four boundary crossings. Each with its own trigger, transformation, consistency contract, and failure mode.

What Each Model Guarantees — and Delegates

Model	Engine enforces	Application must enforce
Relational	FK existence, CHECK, UNIQUE, ACID atomicity — on every write	Nothing (if constraints are complete)
XML/XSD	Sequence, types, cardinality — at validation time	Invoking the validator
Property Graph	Node/edge existence (MERGE creates if absent)	Referential integrity, constraint predicates
Document	Single-doc atomicity; optional JSON Schema	Cross-doc integrity, referential consistency
RDF	Structural correctness of triples (S-P-O)	Everything else — types, cardinality, referential integrity

As data flows left-to-right on the enforcement spectrum, the burden shifts from the **engine** to the **application**.

One Question, Five Languages

"Find all rides by driver Ahmad Ibrahim where the fare exceeded SGD 60."

Language	Can Answer?	What It Returns	What It Misses
SQL	**Yes** — authoritative	All rides with fare > SGD 60, with payment details	N/A (source of truth)
MongoDB	Partially — shape mismatch	Would need cross-collection query	Rides not in driver's aggregate
Cypher	Partially — threshold filtered	Only transfers > SGD 60 threshold (from edges that exceed the SGD 50 creation threshold)	Rides under SGD 50 never entered graph
SPARQL	**No** — wrong store	Zone and licence data only	Ride data not in RDF graph
XPath	Partially — different granularity	Monthly settlement totals, not individual ride fares	Aggregated; monthly batch

SQL (authoritative — relational is the source of truth for ride data):

```
SELECT r.ride_id, r.fare, r.ride_date, p.amount
FROM Rides r JOIN Payments p ON r.ride_id = p.ride_id
  JOIN Drivers d ON r.driver_id = d.driver_id
WHERE d.name = 'Ahmad Ibrahim' AND r.fare > 60;
```

Cypher (threshold filtered — only transfers > SGD 50 entered the graph):

```
MATCH (pax)-[t:TRANSFERRED_TO]→(ahmad:Account {id: 'DRV-5582'})
WHERE t.amount > 60 AND t.transfer_type = 'ride_payment'
RETURN pax.id, t.amount, t.date
-- Missing: rides ≤ SGD 50 never created edges; rides SGD 50–60
in graph but filtered out
```

XPath (different granularity — monthly batch, not individual rides):

```
//Invoice[Customer/CompanyID = 'SG-S1234567A']/InvoiceLines/InvoiceLine[LineAmount > 258.50]
-- Returns monthly settlement totals, not individual ride fares.
```

MongoDB (shape mismatch — Ahmad's profile doesn't store rides *by* him):

```
// Ahmad's document stores his profile, not his ride history.
// Rides live in the relational Rides table or as recentRides
// entries in passenger profiles — wrong aggregate boundary.
```

SPARQL (wrong store — RDF holds identity/zones, not ride data):

```
PREFIX ex: <http://example.org/ridelink#>
SELECT ?driverName ?zone
WHERE {
  ex:driver_drv5582 rdfs:label ?driverName ;
                    ex:operatesIn ?zone .
}
-- Returns zones, not rides. Ride data is not in the RDF graph.
```

The same question gets a different answer from every store — because each stores **different aspects** for **different workloads**. That is the design working correctly.

The Systematic Procedure: Eight Steps

Decomposition (Steps 1–3):

1. **List** the system's services — data owned, operations performed
2. **Group** into workloads by shared access pattern and consistency requirement
3. **Characterise** each workload — fill in the four-property workload card

Assignment and trade-offs (Steps 4–5):

1. **Assign** the best-fit model — is the dominant operation on the cheap path?
2. **State** the trade-off — what is cheap, what is expensive if substituted?

Boundary design and validation (Steps 6–8):

1. **Identify boundaries** — trigger, source of truth, transformation, consistency, failure mode
2. **Draw** the architecture diagram — stores, workloads, data-flow arrows
3. **Validate by substitution** — swap a model, trace what breaks. If nothing breaks, consider consolidating

Procedure Applied: Super-App Decomposition

Workload	Model	What's cheap	Cost if substituted
Transactional core	**Relational**	ACID across 4 tables; FK + CHECK on every write	Document: no FK safety net; promo limit app-managed
Personalisation	**Document**	Aggregate read in 1 call; structural variation without migration	Relational: 4-table JOIN; ALTER TABLE for new roles
Regulatory reporting	**XML/XSD**	Schema-validated exchange; tax authority mandates XML	JSON: no element ordering; portal rejects non-XML
Partner integration	**RDF**	URI identity resolves cross-org mismatch; SPARQL federates	Relational: ad-hoc ETL; breaks on format changes
Fraud monitoring	**Property graph**	3-hop traversal in 6 lines; cycle detection native	Relational: recursive CTE ~20 lines; string-hack cycles

Every assignment has a two-part justification: what the chosen model makes cheap **and** what a substitute would make expensive.

Check 1: "One System = One Workload"

A hospital: patient admissions (ACID transactions), radiology image management (aggregate-shaped metadata reads), clinical research integration (merging patient datasets from three hospitals with different coding systems).

A student says: "I have one workload: the hospital system."

> **What is wrong?** At least three distinct access patterns — relational, document, RDF. Collapsing them hides the model-selection decision. If all workloads are treated as one, the designer is forced into a single model — reproducing **Failure 1**. Decomposition (Steps 1–3) is where the design begins.

Check 2: "Schema Validation = XML/XSD"

A student assigns XML/XSD to the personalisation workload "because we need schema validation on profiles."

> **Why is this wrong?** Schema validation on profiles is an internal concern — MongoDB's JSON Schema or application-level validation handles it. XML/XSD is for cross-organisational exchange where the schema is **externally imposed**. Step 4 (assignment) requires checking the workload's dominant operation against the model's cheap path. Internal profile reads are aggregate-shaped — that is the document model's cheap path. Topic 7's trust-boundary distinction: XML/XSD is right when the contract crosses an organisational boundary with low trust.

Check 3: "Triples Can Represent Anything"

A student assigns RDF to the transactional core because "triples can represent anything."

What mechanisms from Topics 2–5 does this sacrifice?

- ACID transaction isolation (Topic 5)
- FK constraint enforcement (Topic 2)
- CHECK constraints (Topic 5)
- Normalised single-source-of-truth storage (Topic 3)

RDF has no native transaction model, no constraint enforcement, and open-world semantics — the **exact opposite** of transactional requirements.

> **Representational generality is not operational fitness.** Any model can *store* any data. Step 4 (assignment) asks which model's cheap path aligns with the workload's dominant operations — not which model can represent the data. That distinction is arguably the core lesson of the entire course.

Check 4: "All Boundaries Should Be ACID"

A logistics company: relational for shipment tracking, document for warehouse dashboards, property graph for route optimisation. A student labels every data-flow arrow "ACID."

> **Why is this unrealistic?** Step 6 (boundary design) requires specifying the consistency contract per crossing. A shipment status triggering a dashboard refresh should be **eventual** — a 5-second delay is acceptable and far cheaper than a distributed transaction. Road-closure data ingested nightly is **batch** with known staleness. Each boundary's consistency contract matches the workload's **actual tolerance**, not an aspirational ideal.

Check 5: "Add All Five Models for Completeness"

A local bookshop: orders (ACID), product catalogue with reviews (aggregate reads), quarterly tax reports (mandated XML). The developer also adds RDF for "future partner integration" and a property graph for "potential fraud detection."

No current partners. No fraud history.

> **Remove both.** "Future" requirements are not workloads — they have no dominant operations, no consistency requirements, no growth patterns. But the cost is real: every additional model adds at least one boundary, and boundaries are where operational complexity concentrates.

Cost: correctness risk, developer hours, operational burden. **Benefit:** none until the workload exists. The consolidation check (Step 8) catches this.

Check 6: "No Arrows Needed"

A student's architecture diagram has five data stores — relational, document, property graph, RDF, XML — but **no data-flow arrows** between them.

> **What is missing?** The boundaries. A completed ride must flow from relational to document profile, possibly to the fraud graph, eventually to invoicing. Without arrows, the design assumes each store is independent — and data silently diverges. This is **Failure 2** revisited. An architecture diagram without data-flow arrows is a collection of independent databases, not an integrated system. Step 6 exists to prevent this.

Check 7: "ID Migration Is a Local Change"

A super-app team migrates `driver_id` from auto-increment integer to UUID. The document profiles, property graph nodes, and RDF URIs all embed the old integer-based ID. The XML invoices use the national registry ID.

> **What breaks?** Three of four boundaries. Document profiles referencing `driver-5582` point to a non-existent relational ID. Graph nodes with `DRV-5582` are orphaned. RDF URIs embedding `drv5582` lose their link. **XML is unaffected** — it uses `SG-S1234567A`, independent of the internal scheme.

Mitigation: Design a canonical identifier independent of relational storage format — or accept that ID migrations require coordinated updates across all stores. This is the integration tax from Slide 18, made painful.

Seven Misconceptions: The Pattern

#	Misconception	Domain	Procedure step challenged
1	"One system = one workload"	Hospital	Steps 1–3 (decomposition)
2	"Schema validation = XML/XSD"	Super-app	Step 4 (assignment)
3	"Triples can represent anything"	Super-app	Step 4 (fitness inversion)
4	"All boundaries should be ACID"	Logistics	Step 6 (consistency)
5	"Add all five for completeness"	Bookshop	Step 8 (consolidation)
6	"No arrows needed"	Generic	Step 6 (boundaries)
7	"ID migration is a local change"	Super-app	Step 6 (identifier alignment)

Each targets a different step and a different domain. The common thread: multi-model design is not about choosing the "best" model — it is about matching models to workloads, designing boundaries, and being honest about costs.

Revisiting the Failures

Earlier we asked: what **five capabilities** must a multi-model design provide to prevent both failures? Now you have the vocabulary — and the procedure — to answer precisely:

Failure	Diagnosis	Prevention
Single-model trap	Workload-model mismatch: 4 workloads on one model's expensive path	Steps 1–5 — decompose and assign
Right models, no boundaries	Missing boundary design: stores diverge without data-flow specification	Steps 6–7 — design boundaries

The eight-step procedure would have caught both.

Course Arc: From Single Table to Multi-Model Architecture

Ch	Model	Core contribution	What it adds to the capstone
1	All five (overview)	Decision framework, vocabulary	Framework applied five times at system scale
2–5	Relational	ERD → normalisation → SQL → enforcement	FK, CHECK, ACID govern the transactional core
6	Document	Aggregate design, snapshot discipline	Personalisation layer becomes one justified component
7	XML/XSD	Schema-on-write at org boundary	Regulatory reporting boundary with XSD validation
8–9	RDF + SPARQL	URI identity, cross-source integration	Partner integration via URI alignment
10	Property graph	Bounded traversal, cycle detection	Fraud monitoring component
11	**All five (synthesis)**	**Decompose, assign, design boundaries, test**	**This chapter**

The progression is intentional. Each chapter opened a thread. This chapter closes them all.

Vocabulary Evolution: Threads That Close Here

Term	Introduced	Deepened	Capstone meaning
Schema	Topic 1 (DDL)	Topic 6 (schema-on-read), Topic 7 (XSD), Topic 8 (ontology)	Model-specific: DDL, JSON Schema, XSD, OWL, node labels
Constraint	Topic 2 (FK)	Topic 5 (CHECK/UNIQUE/ACID), Topic 7 (xs:sequence)	Enforcement spectrum — engine-enforced to descriptive
Workload	Topic 1 (access pattern)	Topic 3 (OLTP/OLAP), Topic 6 (profile load)	Characterised card: shape, ops, consistency, growth
Snapshot	Topic 3 (shipping address)	Topic 6 (subscriptionSnapshot)	Historical fact at write time, insulated from changes
(min, max) reading	Topic 2 (ERD cardinality)	Topic 7 (minOccurs/maxOccurs), Topic 8 (ontology cardinality)	Same "how many?" skill across notation systems

Course Map

```
Part 1 — Relational Foundation
├── Ch 1  Introduction to Data Models
├── Ch 2  Conceptual Modelling (ERD)
├── Ch 3  Normalisation (BCNF & Star Schemas)
├── Ch 4  SQL Querying (FWGHOS Pipeline)
└── Ch 5  Enforcement (Constraints, Access Control, Transactions, Indexes)

Part 2 — Alternative Models
├── Ch 6  Document Model (JSON & MongoDB)
├── Ch 7  XML for Contracts (XSD & XPath)
├── Ch 8  RDF and Knowledge Graphs
├── Ch 9  SPARQL Querying
└── Ch 10 Property Graphs (Cypher)

Part 3 — Capstone
└── Ch 11 Multi-Model Design ◄── you are here
```

The five data models are not competitors. They are tools for different jobs. The engineer's skill is matching each tool to its workload, designing the boundaries between them, and stating the trade-offs honestly.

The Course in One Sentence

> The five data models are not competitors — they are tools for different jobs, and the engineer's skill is matching each tool to its workload, designing the boundaries between them, and stating the trade-offs honestly.

The design maturity demonstrated here — decomposing by workload, assigning by fit, designing boundaries, testing by substitution — is the skill that distinguishes a **database user** from a **database engineer**.

Lab Preview

Lab 11 — Multi-Model Capstone: Designing a Super-App Data Architecture

A design lab, not a coding lab — architectural reasoning, not query syntax.

- **Part 1** (~20 min): System decomposition — group 8 services into workloads
- **Part 2** (~40 min): Model assignment with two-part justifications
- **Part 3** (~25 min): Data-flow boundary design with consistency contracts
- **Part 4** (~30 min): Architecture diagram with annotated arrows
- **Part 5** (~20 min): Substitution testing — swap models, trace what breaks
- **Part 6** (~30 min): Cross-model query analysis
- **Part 7** (~15 min): Reflection — monolith vs multi-model trade-offs

Format: Written design analysis. **Total ~180 min.**

File: `11-lab-multi-model-capstone.md`